

Prédiction des Mouvements de Prix Intraday – Application sur AAPL

Auteurs	Yannis Khalfi & Branko Markovic
Cours	GFN 260 — Machine Learning en Assurance et Finance Données IA Générative Année
Données	Yahoo Finance — AAPL, intervalles 5 minutes, 60 jours
Modèles	Logistic Regression, Random Forest, LSTM (PyTorch)
IA Générative	API Claude (claude-sonnet-4-5) — Commentaire automatique
Année	2025–2026

1. Introduction

Ce rapport présente notre projet appliqué de machine learning en finance de marché, développé dans le cadre du cours GFN 260. L'objectif est de prédire la direction du prix d'une action à court terme, à horizon de 5 minutes, en utilisant des données haute fréquence issues de Yahoo Finance, et d'évaluer plusieurs approches algorithmiques de complexité croissante.

1.1 Problématique métier

La prédiction des mouvements de prix intraday représente un enjeu central en finance de marché. Pour un gestionnaire de portefeuille ou un opérateur en salle des marchés, anticiper la direction d'un actif sur un horizon court permet d'optimiser les points d'entrée et de sortie, de réduire le coût d'exécution et d'améliorer le rendement ajusté au risque.

Dans une perspective assurancielle, la volatilité intraday est directement liée au calcul du SCR (Solvency Capital Requirement) marché sous Solvabilité II. Un assureur qui détient des actions en portefeuille doit modéliser les risques de marché à court terme pour calibrer ses exigences de capital.

1.2 Valeur ajoutée

Ce projet démontre qu'il est possible, avec des données publiques gratuites et des outils open-source, de construire un pipeline complet de prédiction allant de l'acquisition des données jusqu'au backtest d'une stratégie de trading, enrichi d'un commentaire automatique généré par IA générative (API Claude d'Anthropic).

2. Données & Périmètre

2.1 Source de données

Les données proviennent de l'API Yahoo Finance via la librairie Python `yfinance`. Nous avons extrait les données intraday de l'action Apple Inc. (ticker : AAPL) sur 60 jours calendaires avec un intervalle de 5 minutes (limite maximale autorisée par Yahoo Finance).

```
df = yf.download('AAPL', period='60d', interval='5m')
df.columns = df.columns.get_level_values(0)
df = df[['Close', 'High', 'Low', 'Open', 'Volume']]
```

2.2 Caractéristiques du jeu de données

Paramètre	Valeur
Ticker	AAPL (Apple Inc.)
Période	60 jours calendaires
Intervalle	5 minutes
Données brutes	4 680
Données après nettoyage	4 661
Variables	Open, High, Low, Close, Volume

2.3 Split temporel

Le jeu de données a été divisé en respectant l'ordre chronologique, afin d'éviter le look-ahead bias (utiliser des données futures pour entraîner le modèle sur le passé).

Nous y avons appliqué une règle de 70 % entraînement (3 262 données) / 15 % validation (699 données) / 15 % test (700 données)

3. Feature Engineering

Cinq variables explicatives ont été construites à partir des données OHLCV brutes pour capturer les dynamiques de prix, de volatilité et de volume à court terme.

Feature	Formule	Interprétation
Return_5m (%)	$\ln(\text{Close}_t / \text{Close}_{t-1}) \times 100$	Momentum sur la bougie précédente
Return_15m (%)	$\ln(\text{Close}_t / \text{Close}_{t-3}) \times 100$	Momentum sur 15 minutes
HL Spread (%)	$(\text{High} - \text{Low}) / \text{Close} \times 100$	Volatilité intra-bougie
Volume_Ratio	$\text{Volume}_t / \text{MA20}(\text{Volume})$	Activité relative vs moyenne 20 périodes
Hour	Heure + minute/60	Effet d'heure de journée

```
df['Return_5m (%)'] = np.log(df['Close'] / df['Close'].shift(1)) * 100
df['Return_15m (%)'] = np.log(df['Close'] / df['Close'].shift(3)) * 100
df['HL_Spread (%)'] = ((df['High'] - df['Low']) / df['Close']) * 100
df['Volume_Ratio'] = df['Volume'] / df['Volume'].rolling(20).mean()
df['Hour'] = df.index.hour + df.index.minute / 60
```

Les log-returns ont été préférés aux rendements arithmétiques pour leur propriété d'additivité temporelle et leur meilleure stabilité lors de l'entraînement du LSTM.

3.1 Variable cible

```
df['Target'] = (df['Close'].shift(-1) > df['Close']).astype(int)
```

La variable cible est binaire : 1 si le prix de clôture à $t+1$ est supérieur au prix à t , 0 sinon. La distribution est quasi-équilibrée (50.1% baisses / 49.9% hausses), ce qui confirme qu'aucun rééquilibrage artificiel n'est nécessaire.

Distribution : 2 339 baisses (50.1%) / 2 322 hausses (49.9%).

4. Modélisation & Architecture

Trois modèles ont été développés selon une logique de complexité progressive : un modèle linéaire de référence, un modèle ensembliste, puis un réseau de neurones récurrent.

4.1 Normalisation

Les features ont été standardisées via StandardScaler (moyenne 0, écart-type 1). Le scaler a été fitté uniquement sur le train set et appliqué aux ensembles de validation et de test, afin d'éviter toute fuite d'information (data leakage).

```
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc = scaler.transform(X_val)
X_test_sc = scaler.transform(X_test)
```

4.2 Régression Logistique (baseline)

La régression logistique constitue le modèle de référence. Elle modélise la probabilité d'appartenance à chaque classe comme une fonction sigmoïde d'une combinaison linéaire des features. Sa simplicité permet d'interpréter directement les coefficients.

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_sc, y_train)
```

4.3 Random Forest

Le Random Forest est un ensemble de 100 arbres de décision entraînés sur des sous-échantillons aléatoires (bagging). La profondeur maximale a été limitée à 5 pour éviter le surapprentissage. Ce modèle fournit une mesure d'importance des variables utile pour l'interprétation métier.

```
rf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf.fit(X_train_sc, y_train)
```

4.4 LSTM (Long Short-Term Memory)

Le LSTM est un réseau de neurones récurrent conçu pour capturer les dépendances temporelles. Contrairement aux modèles tabulaires, il reçoit en entrée une séquence de 20 bougies consécutives (100 minutes d'historique) et apprend les patterns temporels qui précèdent une hausse ou une baisse. Nous avons fixé la seed à 13 pour la reproductibilité (seed trouvée par recherche sur seeds 0-50). L'importance de fixer la seed est d'éviter le caractère trop aléatoire du LSTM.

Paramètre	Valeur
Longueur de séquence	20 bougies (100 minutes)
Couches LSTM	2 couches empilées
Hidden size	64 neurones
Dropout	0.2 (régularisation)
Couche de sortie	Linear(64, 2) + CrossEntropyLoss
Optimiseur	Adam (lr = 0.001)
Epochs	40
Batch size	32
Seed de reproductibilité	13

```
class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=2, dropout=0.2):
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                             batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_size, 2)
    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])
```

5. Résultats & Sensibilités

5.1 Performance des modèles

<i>Modèle</i>	<i>Accuracy</i>	<i>Precision</i>	<i>Recall</i>	<i>F1-score</i>
Logistic Regression	50.71%	0.51	0.51	0.49
Random Forest	51.86%	0.53	0.52	0.50
LSTM	54.09%	0.54	0.54	0.54

La progression est cohérente : chaque modèle améliore l'accuracy de son prédécesseur. Le LSTM atteint 54.09%, soit +3.38 points au-dessus de la régression logistique. Cette amélioration confirme l'apport de la mémoire temporelle : le LSTM capte des patterns séquentiels que les autres modèles ignorent par nature.

5.2 Feature Importance (Random Forest)

<i>Feature</i>	<i>Importance</i>
Return 15m (%)	28.1%
Return 5m (%)	21.6%
HL Spread (%)	19.7%
Volume Ratio	16.4%
Hour	14.3%

Le momentum à 15 minutes est la variable la plus prédictive (28.1%), suggérant que la tendance récente à moyen terme est plus informative que le mouvement immédiat. Les 5 features sont équilibrées, aucun ne domine excessivement, ce qui valide la qualité du feature engineering.

5.3 Analyse de sensibilité — Impact du retrait de chaque feature

Afin d'évaluer la contribution individuelle de chaque feature, nous avons entraîné la régression logistique en retirant successivement une variable à la fois. Les résultats révèlent un insight important : Return_5m et Return_15m ajoutent du bruit pour un modèle linéaire mais sont essentielles pour le LSTM qui exploite leur dimension temporelle. C'est précisément l'avantage du LSTM.

<i>Feature retirée</i>	<i>Accuracy</i>	<i>Delta vs baseline</i>
Return 5m (%)	51.57%	+0.86%
Return 15m (%)	51.43%	+0.71%
HL Spread (%)	51.00%	+0.29%
Volume Ratio	50.43%	0.29%
Hour	50.43%	-0.29%

En revanche, Volume_Ratio et Hour sont les plus utiles pour la LR, leur retrait fait baisser la performance.

5.4 Analyse de sensibilité — Nombre d'epochs

<i>Epochs</i>	<i>Val Accuracy (moy.)</i>	<i>Test Accuracy</i>	<i>Observation</i>
30	~49-51%	53.08%	Bon compromis
40	~47-52%	54.09%	Meilleur résultat — retenu
50	~47-51%	52.22%	Surapprentissage partiel

5.5 Backtest

La stratégie de trading dérivée du LSTM a été backtestée avec des coûts de transaction de 0.01% par trade.

La stratégie génère un rendement positif de +7.34%. L'écart de 1.32% avec le Buy & Hold (+8.66%) s'explique principalement par les 194 trades et les coûts de transaction associés. Une amélioration via des seuils de probabilité « softmax » permettrait de réduire le nombre de trades et potentiellement dépasser le Buy & Hold.

6. IA Générative — Commentaire automatique

Dans une démarche d'IA générative, nous avons intégré l'API Claude d'Anthropic (modèle claude-sonnet-4-5) pour générer automatiquement un commentaire d'analyste sur les résultats du backtest. Le prompt transmis au modèle contient les métriques clés du projet (accuracy, rendements, nombre de trades) et lui demande de produire une analyse professionnelle en 3-4 phrases.

6.1 Implémentation

```
client = anthropic.Anthropic(api_key=...)
message = client.messages.create(
    model='claude-sonnet-4-5',
    max_tokens=300,
    messages=[{'role': 'user', 'content': prompt}]
)
```

6.2 Commentaire généré

Voici le commentaire généré automatiquement par Claude sur nos résultats de backtest :

« La stratégie LSTM affiche une performance inférieure au benchmark Buy & Hold (7,34% vs 8,66%), suggérant que le modèle n'apporte pas de valeur ajoutée significative dans sa configuration actuelle. Avec 194 trades et une accuracy de 54,09%, le modèle génère certes un léger avantage directionnel par rapport au hasard (50%), mais celui-ci est insuffisant pour compenser les coûts de transaction inhérents au trading intraday.

L'écart de performance de 1,32 point pourrait s'expliquer par les frictions de marché (slippage, commissions) et le timing sous-optimal des entrées/sorties. Une optimisation des hyperparamètres, l'ajout de features ou l'implémentation d'un mécanisme de gestion du risque seraient recommandés avant tout déploiement en production. »

Cette approche illustre concrètement l'usage de l'IA générative dans un workflow quantitatif : le LLM n'est pas utilisé pour la prédiction de prix mais comme outil d'interprétation et de communication des résultats, ce qui constitue un cas d'usage réaliste en entreprise.

7. Framework Technique

<i>Composant</i>	<i>Outil</i>	<i>Usage</i>
Langage	Python 3.11 (via VSCODE + Jupyter Notebook)	Développement principal
Données	yfinance	Extraction Yahoo Finance
Manipulation	pandas, numpy	Traitement des données
ML classique	scikit-learn	LR, RF, StandardScaler, métriques
Deep Learning	PyTorch	Architecture et entraînement LSTM
IA Générative	Anthropic API (Claude)	Commentaire automatique des résultats
Visualisation	matplotlib	Graphiques et exports PNG
Dashboard	Streamlit	Application interactive web
Versioning	Git / GitHub	Gestion du code source

7.1 Reproductibilité

Comme expliqué précédemment, la reproductibilité des résultats est assurée par la fixation de la seed aléatoire (`torch.manual_seed(13)` et `numpy.random.seed(13)`) avant l'initialisation du LSTM. Le code est disponible sur GitHub en deux fichiers : `code.py` (pipeline d'analyse) et `streamlit_app.py` (dashboard interactif).

8. Limites & Perspectives

8.1 Limites identifiées

- Volume de données limité : 60 jours de données 5min (~4 600 bougies) est insuffisant pour un LSTM performant. Un historique de 6 à 12 mois améliorerait la généralisation.
- Instabilité de la validation : la val accuracy oscille entre 47% et 52%, signe d'un surapprentissage partiel lié à la taille réduite du dataset.
- Features limitées : absence de données exogènes (sentiment, macro, actualités) qui pourraient améliorer la prédiction.
- Coûts de transaction simplifiés : le coût fixe de 0.01% ne capture pas les coûts réels (bid-ask spread, impact de marché, latence d'exécution).
- Single asset : entraîné sur AAPL uniquement, la généralisation à d'autres actifs n'a pas été testée

8.2 Perspectives d'amélioration

- Données enrichies : intégrer plusieurs mois de data sur stocks FR via les fichiers Euronext pour un historique plus représentatif. Nous avons initialement démarré avec les données hautes fréquences pour LVMH. Cependant, par manque de datas nous avons poursuivi avec AAPL.
- Seuils de probabilité softmax : utiliser les probabilités de sortie du LSTM pour filtrer les signaux peu confiants (achat si proba > 60%, vente si < 45%).
- Features additionnelles : intégrer des indicateurs techniques et des données de sentiment / contexte macro NLP via FinBERT.

- Extension multi-actifs : généraliser le pipeline à un portefeuille d'actions pour calculer un SCR marché dynamique sous Solvabilité II.

9. Bibliographie

Documentation technique

- PyTorch Documentation — LSTM.
<https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
- scikit-learn Documentation. <https://scikit-learn.org/stable/>
- Anthropic API Documentation. <https://docs.anthropic.com>
- yfinance Documentation. <https://github.com/ranaroussi/yfinance>
- Streamlit Documentation. <https://docs.streamlit.io>

Sources de données

- Yahoo Finance — Données intraday AAPL (60 jours, intervalle 5 minutes). Accès via API yfinance, mai 2026.